DigitalLending (DLCC)

Digital Lending on Credit Card: 5 web APIs, from loan quote to disbursal check

This guide explains the loan journey, the business rules in each API, how the project shares the ILCC batch pipeline, and what in the supplied code is mocked. Read it together with the ILCC guide: the two projects are close cousins.

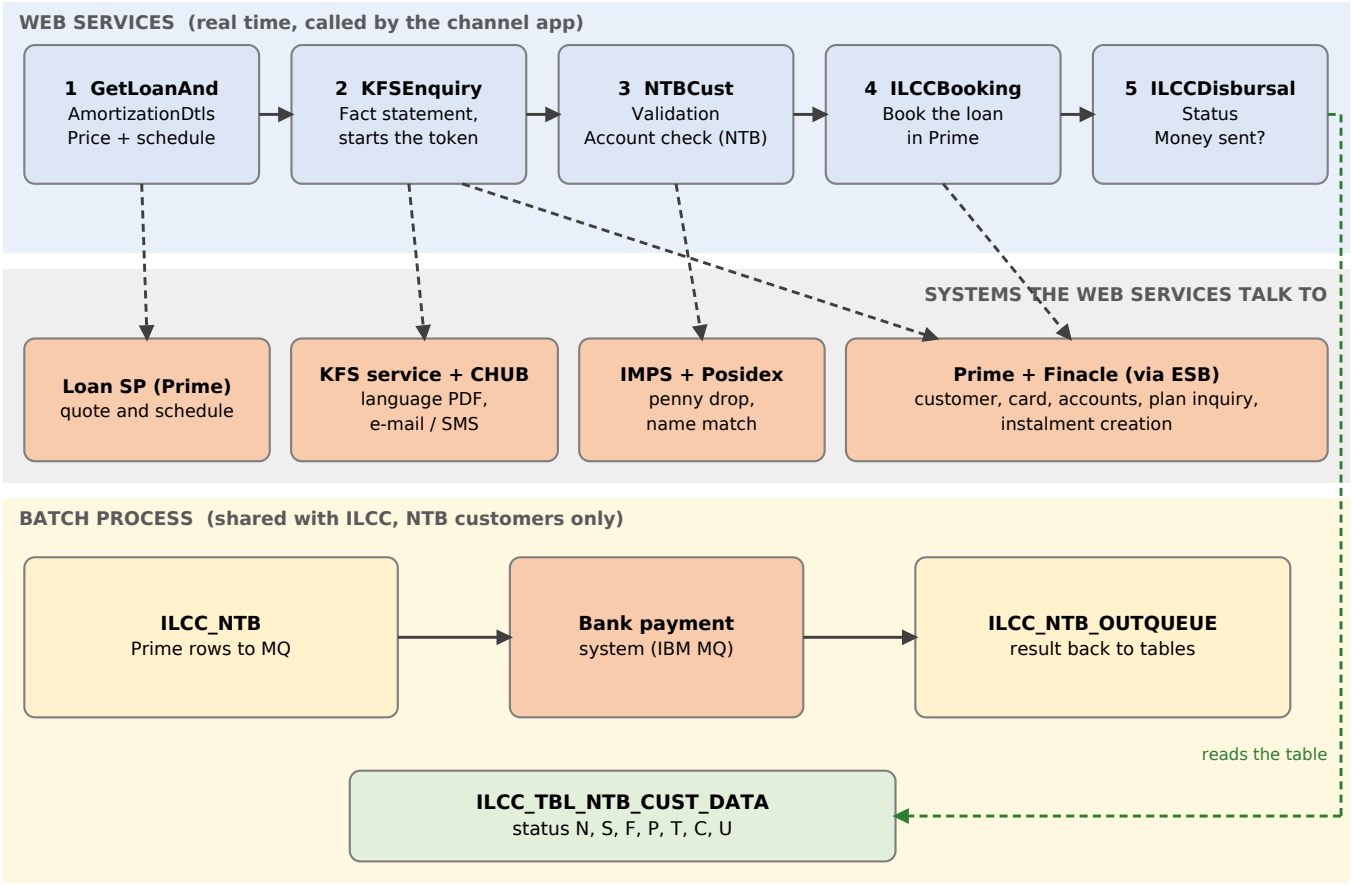
Built from the DigitalLending web code. The batch part is shared with ILCC (see section 5).

1. DigitalLending in plain words

A cardholder picks a loan amount and tenure on the credit card, sees the full cost and the Key Fact Statement (KFS), and then books the loan. The same loan family as ILCC, but this project **starts with a price quote and a regulatory fact sheet** before booking. Five APIs, called in order: **quote, KFS, account check (NTB only), booking, disbursal status**.

Term	Meaning in this project
DLCC	Digital Lending on Credit Card (the name used in the code and tables, prefix dlcc_).
WTL / OTL	**Within the Limit** and **Over the Limit** loans. The code maps instalment type IPPLWTL to WTL and IPPLOTL to OTL.
KFS	Key Fact Statement: the document that tells the customer the loan terms, fees, total cost and repayment schedule before they accept.
Stamp duty / net disbursed	A stamp-duty charge is deducted from the sanctioned amount. Net disbursed amount = sanctioned loan amount minus stamp duty.
ETB / NTB	Existing-to-bank (gets the money in an own Axis account) / New-to-bank (gets NEFT in another bank). Same meaning as in ILCC.
TokenRefId	Id of one customer journey. Starts at KFSEnquiry and is re-used by the next steps.
CardSerNo	Internal serial number of the card in Prime. Main key of all five APIs.
Penny drop, Posidex	Test transfer to prove an account exists and name matching of the holder (as in ILCC).

2. The whole journey at a glance



Read the diagram like this

- **Step 1** only calculates. It stores the quote so that step 2 can build the KFS from it.
- **Step 2 opens the journey.** Its TokenRefId must be new. It also finds the customer (ETB or NTB) and creates the master row.
- **Step 3** is only for NTB customers. **Step 4** books the loan. **Step 5** reads the batch tables.
- The **batch jobs are the ILCC ones**. DigitalLending reads the same disbursal tables but has no batch jobs of its own.

3. Rules common to all five APIs

WEB step 1	The request arrives as an encrypted JSON body. It is decrypted and the service name is read from `Request.Body`.
CHECK step 2	**Authenticate the channel.** Allowed channels: **Test, DBAT, MBB, IBB, SIDDHI, DCOM**.
CHECK step 3	**Validate each field** with a pattern (section 4 lists them).
CHECK step 4	**Reject a repeated ReferenceId** (code 101) using dlcc_tbl_request_details.
CHECK step 5	**Check the token** according to the API (table below).
DB step 6	**Log** the request in dlcc_tbl_request_details and each external call in dlcc_Tbl_External_Api_log.
RESULT step 7	Build the response and encrypt it.

How the token (TokenRefId) is checked

API	Token rule
GetLoanAndAmortizationDtls	No token.
KFSEnquiry	Token must be **new**. This call starts the journey.
NTBCustValidation	Token must **already exist** (created by KFSEnquiry).
ILCCBooking	Token must **already exist**.
ILCCDisbursalStatus	No token. Uses CardSerNo and INSTLRefNo.

4. The web APIs one by one

4.1 GetLoanAndAmortizationDtls: price the loan

What it does: calculates what the loan will cost and returns the repayment schedule. It does not create anything in Prime.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
Periods	Mandatory	`^[0-9]{1,2}$`
LoanAmt	Mandatory	`^[1-9][0-9]{3,20}$`
RateOfInterest	Mandatory	`^[0-9]{1,3}(\.[0-9]{1,2})?$`
ProcFee	Mandatory	`^[0-9]{3,10}$`
ProcFeePercent	Mandatory	`^[0-9]{1,3}(\.[0-9]{1,2})?$`
ForcFee	Mandatory	`^[0-9]{3,10}$`
ForcFeePercent	Mandatory	`^[0-9]{1,3}(\.[0-9]{1,2})?$`
InterestReasonCode	Mandatory	`^[A-Za-z]{4}$`
InstallmentType	Mandatory	`^[A-Za-z]{4,10}$`

Business flow

WEB step 1	Common checks (no token). CardSerNo must be present.
CALL step 2	**Run the loan stored procedure** with: instalment type, periods, loan amount, processing fee, rate of interest, processing-fee percent, interest reason code, card serial number. It returns one summary row and the list of instalments.
CHECK step 3	**Procedure result not "Success"?** Return an external-system failure with the Prime code and text.
CHECK step 4	**Work out the figures.** Loan type text: IPPLWTL gives "Within the Limit (WTL)", IPPLOTL gives "Over the Limit (OTL)". Stamp duty is shown as a charge. **Net disbursed = loan amount - stamp duty. Amount to be paid = loan amount + total interest + processing fee.** Initial-period interest is shown separately.
CHECK step 5	**Fill the templates**: loan details block (type, "rate % p.a. (Fixed)", fees, interest), fee and payment block, and the repayment schedule (one entry per instalment).
DB step 6	**Save the quote** in dlcc_tbl_kfsloan_details (REF_ID, request, response, CARDSERNO). KFSEnquiry reads it from here.
RESULT step 7	Return LoanDtls, fee details and RepaymentSchedule.

Watch out (supplied code). `execute_loan_sp` returns mock data (`variable_out`, `cursor_out`) and the real call is not wired up (an unreachable `NotImplementedError` says "testing_mode is required for now"). Every quote in this version comes from the mock.

4.2 KFSEnquiry: the Key Fact Statement

What it does: returns the KFS for the quote the customer saw. It also opens the customer journey and works out whether the customer is ETB or NTB.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
TokenRefId	Mandatory	`^[0-9a-zA-Z]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
VernacLang	Optional	`^[A-Za-z]{1,15}$`

Supported VernacLang values: ENGLISH, HINDI, GUJARATI, ASSAMESE, MARATHI, KANNADA, BENGALI, MALAYALAM, ORIYA, TAMIL, TELUGU, PUNJABI. Anything else gives a "language not found" error.

Business flow

WEB step 1	Common checks. The token must be **new**. A non-empty VernacLang must be in the list above.
DB step 2	**Read the stored quote** for this CardSerNo from dlcc_tbl_kfsloan_details. None: error "CardSerNo not exist".
CALL step 3	**Find the customer** (same three-step pattern as ILCC): CreditCardESBService turns the serial number into the card number and mobile; Prime EntityInquiry returns customer id, accounts, PAN and date of birth; Finacle get-account-details decides **ETB** (account with a match) or **NTB**.
DB step 4	**Create the master row** in dlcc_Tbl_Master_Details: REF_ID, TOKEN_REF_ID, CARD_SER_NO, CUST_TYPE, account number and customer details.
CHECK step 5	**Language.** English: the stored quote is returned as the KFS data. Any other language: go to step 5.
CALL step 6	**Language KFS (non-English).** Call ESB **Initiate KFS** with the reference, language, customer id, PAN, date of birth, mobile, account number, loan type and the repayment schedule (all base64). **Wait 15 seconds**, then call **KFS Download**. The reply carries the PDF as base64.
CALL step 7	**Send it to the customer.** Get e-mail and mobile with getDOBMailingAddr, then send them the document through CHUB.
RESULT step 8	Return LoanDtls plus the KFS (Base64EncodedString for a language PDF). Failures return external-system failure (222) with the channel name (KFS or PRIME).

Watch out (supplied code). Every one of these external calls (card number, entity inquiry, Finacle, Initiate KFS, KFS Download, mailing address, CHUB) has its real `send_request` commented out and uses a `raw_*` mock reply. The **15-second wait** still runs, so a language KFS takes at least 15 seconds.

4.3 NTBCustValidation: check the bank account (NTB only)

What it does: proves that the NTB customer's other-bank account exists and belongs to the same person. Same logic as ILCC.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
TokenRefId	Mandatory	`^[0-9a-zA-Z]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
AccNo	Mandatory	`^[0-9]{15,20}$`
IFSC	Mandatory	`^[0-9a-zA-Z]{10,12}$`

Business flow

WEB step 1	Common checks. The token must already exist.
DB step 2	Read the master row (customer name) by TokenRefId.
CALL step 3	**IMPS penny drop** to AccNo + IFSC through the internal IMPS service. The bank returns PennyDropStatus and the account holder name.
CHECK step 4	**Penny drop not Y?** Return PennyDropStatus N and NameMatchStatus N. No name match is tried.
CALL step 5	**Posidex name match** through ESB: card holder name against the penny-drop name.
CHECK step 6	**75 percent or more:** NameMatchStatus **Y**. Below 75: **N**. Empty percentage while a name exists: external failure.
DB step 7	Update the master row and log both calls.
RESULT step 8	Return CardSerNo, PennyDropStatus, NameMatchStatus.

Watch out (supplied code). The IMPS call is live here (the only live call in this project). The Posidex reply is a mock (`raw_namematch_response`).

4.4 ILCCBooking: book the loan

What it does: creates the instalment (the loan) in Prime. Same pattern as ILCC; the API keeps the name ILCCBooking.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
TokenRefId	Mandatory	`^[0-9a-zA-Z]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
Tenure	Mandatory	`^[0-9]{1,4}$`
LoanAmnt	Mandatory	`^[0-9]{1,10}$`
InterestRate	Mandatory	`^[0-9.]{1,5}$`
FCFeeFlat	Mandatory	`^[0-9.]{1,6}$`
ProcFeeFlat	Mandatory	`^[0-9.]{1,6}$`
ProcFeePer	Mandatory	`^[0-9.]{1,3}$`
FCFeePer	Mandatory	`^[0-9.]{1,3}$`
INSTLType	Mandatory	`^[A-Za-z\-]{3,10}$`
AccNo	Mandatory	`^[0-9][0-9X]{7,20}$`

Business flow

WEB step 1	Common checks. The token must already exist.
DB step 2	Read the master row. **ETB without AccNo** stops with "AccNo Empty". Look up the customer bank name from the IFSC (dlcc_tbl_bank_mapping_conf) and read the current Prime ticket.
CALL step 3	**Prime GetInstalmentPlanInquiry** returns the plan code. Code 4 = no plan (102). Other non-zero = external failure (222).
CHECK step 4	**How is the money paid?** ETB: **Account Transfer** to the chosen own account. NTB: **NEFT** to the other bank.
CALL step 5	**Prime InstalmentCreation.** Code 12 = not found. Zero = the instalment exists.
DB step 6	**Update the master row**: AMOUNT and BOOKING_STATUS (**SUCCESS** when Status is Y, else **FAILURE**). A failure also writes FAILURE.
RESULT step 7	Return CardSerNo, Status Y or N, INSTLRefNo (key for step 5) and a description.

Watch out (supplied code). Plan inquiry and instalment creation use mock replies (`raw_plan_inquiry_response`, `raw_plan_creation_response`).

4.5 ILCCDisbursalStatus: has the money been sent?

What it does: shows the state of the loan payment. Designed to read the same places as ILCC.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
INSTLRefNo	Mandatory	`^[0-9]{5,20}$`

Business flow

WEB step 1	Common checks. No token.
DB step 2	Read CUST_TYPE from the master row.
DB step 3	**Designed behaviour:** NTB reads ILCC_TBL_NTB_CUST_DATA (filled by the batch jobs) and turns the status letter into text. ETB reads Prime prm_loan_disb_req. See the status table in the ILCC guide.
RESULT step 4	Return DisbursalStatus (Y or N), INSTLRefNo and Description. No row: 102.

Watch out (supplied code). `get_disbursal_status` does not use the query result: it sets `result_data = [{"code": "000", "status": "Disbursal Successful"}]`. So this version **always answers Y / "Disbursal Successful"** (NTB) or "Disbursal Success" (ETB) once a master row exists. The code notes this copies a default from the old PHP service. Remove it before production, or the channel will show success for loans that were never paid.

5. How DigitalLending connects to the batch process

DigitalLending has **no batch jobs of its own**. Its Query_Conf points at the ILCC tables: NTB disbursal reads **ILCC_TBL_NTB_CUST_DATA**, ETB reads **prm_loan_disb_req** on the Prime link. So the NTB payment pipeline is the one described in the ILCC guide (section 5): ILCC_NTB, ILCC_NTB_OUTQUEUE and KAFKA_CONSUMER. TICKET_GEN keeps the Prime ticket fresh for booking.

Step	Who	Tables
Quote	GetLoanAndAmortizationDtls (web)	writes dlcc_tbl_kfsloan_details
KFS + customer	KFSEnquiry (web)	reads dlcc_tbl_kfsloan_details, writes dlcc_Tbl_Master_Details
Book	ILCCBooking (web)	Prime instalment, updates master row
Pick up, send, result	ILCC_NTB and ILCC_NTB_OUTQUEUE (batch)	PRM_ILCC_LOAN_DISB_REQ, ILCC_TBL_NTB_CUST_DATA, ILCC_TBL_MESSAGE_DETAILS
Show	ILCCDisbursalStatus (web)	reads ILCC_TBL_NTB_CUST_DATA (NTB) / prm_loan_disb_req (ETB)

Please confirm. The shared table names show the batch pipeline *can* serve DigitalLending loans. Whether DigitalLending bookings land in the same Prime request table as ILCC ones is not visible in the web code. Check with the Prime team.

Differences from ILCC

Topic	ILCC	DigitalLending
Starts with	FetchLoanOffer (offers WTL / OTL)	GetLoanAndAmortizationDtls (price quote), then KFSEnquiry
Token starts at	FetchLoanOffer	KFSEnquiry
Regulatory fact sheet	None	KFS, 12 languages, PDF by e-mail/SMS
Customer ETB/NTB lookup	FetchLoanOffer	KFSEnquiry
Tables	ILCC_Tbl_*, Ilcc_Tbl_Temp_Master_Details	dlcc_tbl_*, dlcc_Tbl_Master_Details
Channels	Test, Flipkart CO, MBB, IBB, SOB, SIDDHI, SAK	Test, DBAT, MBB, IBB, SIDDHI, DCOM
State of supplied code	Some mocks, some live Prime calls	Test harness: testing_mode True, almost all calls mocked

6. Reference

External systems

Call	URL (UAT)	Used by	State in supplied code
Prime EntityInquiry	https://esbuat2.axisb.com/json/v2/prime4_0axisisuerapp/EntityInquiry	KFSEnquiry	Mock reply
CreditCardESBService	https://esbuat2.axisb.com/v2/JSON/CreditCardESBService	KFSEnquiry	Mock reply
Finacle account details	https://esbuat2.axisb.com/finacle/account-details/enc/get-account-details	KFSEnquiry	Mock reply
Initiate KFS	https://esbuat2.axisb.com/keyfactstatement/initiation/enc/initiate-kfs	KFSEnquiry (language)	Mock reply
KFS Download	https://esbuat2.axisb.com/keyfactstatement/download/enc/download-kfs	KFSEnquiry (language)	Mock reply
getDOBMailingAddr	https://esbuat2.axisb.com/json/v2/prime4_0prosbapi_2/getDOBMailingAddr	KFSEnquiry (language)	Mock reply

Call	URL (UAT)	Used by	State in supplied code
CHUB send message	https://esbuat2.axisb.com/chub-sms/enc/send-message	KFSEnquiry (language)	Mock reply
IMPS fund transfer	http://192.168.12.85:8020/api/v1/DlccServices	NTBCustValidation	**Live call**
Posidex name match	https://esbuat2.axisb.com/posidex/customernamematch/enc/match	NTBCustValidation	Mock reply
Prime GetInstalmentPlanInquiry	https://esbuat2.axisb.com/json/v2/prime4_0axisapp/GetInstalmentPlanInquiry	ILCCBooking	Mock reply
Prime InstalmentCreation	https://esbuat2.axisb.com/json/v2/prime4_0axisapp/InstalmentCreation	ILCCBooking	Mock reply
Loan stored procedure	(Prime database)	GetLoanAndAmortizationDtls	Mock, not wired up

Also in config but not used by the five APIs: an older IMPS PHP URL (192.168.79.236) and a NeApiService submit URL (192.168.117.150:8180).

Database tables

Table	Used by	Purpose
dlcc_tbl_request_details	All APIs	One row per API call (duplicate check, audit)
dlcc_Tbl_External_Api_log	All APIs	One row per external call
dlcc_tbl_kfsloan_details	Quote (write), KFS (read)	The quote shown to the customer
dlcc_Tbl_Master_Details	APIs 2 to 5	Journey master row, keyed by TokenRefId
dlcc_tbl_bank_mapping_conf	ILCCBooking	IFSC to bank name
Tbl_Ticket_Request_Details	ILCCBooking	Current Prime ticket (batch TICKET_GEN)
ILCC_TBL_NTB_CUST_DATA	Disbursal status (NTB)	NTB payment status, written by the ILCC batch
prm_loan_disb_req (Prime link)	Disbursal status (ETB)	Prime disbursal rows

Response codes (DigitalLending)

Code	Meaning	Code	Meaning
000	Success	001	Authentication Failed
002	Invalid Channel ID	003	Invalid Reference Id
004	Invalid Periods	005	Invalid Loan Amount
006	Invalid Rate Of Interest	007	Invalid Installment Type
008	Invalid Processing Fee	009	Invalid Processing Fee Percentage
010	Invalid Foreclosure Fee	011	Invalid Foreclosure Fee Percentage
012	Invalid Interest Reason Code	013	Invalid Enquiry Reference Id
014	Invalid Token Reference Id	015	Invalid Card SerNo
016	Invalid Account No	017	Invalid IFSC Code
018	Invalid Tenure	019	Invalid Loan Amount
020	Invalid Interest Rate	021	Invalid Processing Fee Flat
022	Invalid Foreclosure Fee Flat	023	Invalid Processing Fee Percent
024	Invalid Foreclosure Fee Percent	025	Invalid Instalment Type
026	Invalid Instalment Ref Number	101	Reference Id Already Exist
102	CardSerNo Not Exist	103	Token Reference Id Not Exist
104	Token Ref Id Already Exist	105	Record Not Found
106	Mandatory values are missing	107	Language Not Found
111	Service Temporarily Unavailable	222	External API Failure
222	External API Failure	999	Unable to Process, Technical Error

Code	Meaning	Code	Meaning
999	Unable to Process, Technical Error		

7. Watch-outs and quick troubleshooting

Things found in the supplied code

- **This is a test harness.** `testing_mode = True` and nearly every external call is a `raw_*` mock. Only the IMPS penny drop is live.
- **Loan quote is mock data.** The real stored-procedure call is not wired up.
- **Disbursal status always says success** (section 4.5). Never release this logic to production.
- **Sensitive data goes to the KFS service.** PAN, date of birth, mobile and account number are sent (base64 encoded, which is not encryption) inside an ESB-encrypted request. Keep logs masked.
- **Slow language KFS.** A fixed 15-second wait sits between Initiate and Download.
- **Secrets.** The config holds AES keys, channel passwords and ESB keys in plain text (masked here).

If something looks wrong

What you see	Where to look first
KFSEnquiry says token already exists (103)	The TokenRefId was used before. A new journey needs a new token.
KFSEnquiry says CardSerNo not exist	GetLoanAndAmortizationDtls was not called first for this card, or its quote row is missing in dlcc_tbl_kfsloan_details.
KFSEnquiry returns language not found	VernacLang is not one of the 12 supported values (upper case).
NTBCustValidation or ILCCBooking says token validation failed	KFSEnquiry was not called first with this token.
ILCCBooking returns "AccNo Empty"	Customer is ETB and AccNo was not sent.
Disbursal status says success but no money moved	Expected in this version (hardcoded success). For the real flow see the ILCC guide, section 8.